



ITM SKILLS
UNIVERSITY

SCHOOL OF FUTURE TECH

PROJECT REPORT

on

Real File Compression and Decompression System

Using C++ and System gzip Utility

Submitted by

Aaryan Kuchekar

Roll Number: 150096725048

Demis Hassabis

2025-2029

[Github Link](#)

Index

1	Introduction to the Project
2	Problem Statement and Abstract
3	Project Design and Architecture
4	Libraries and Header Files Used
5	C++ Concepts Used
6	Complete Code Explanation — Block by Block
6.1	ICompressor — Abstract Base Class
6.2	GzipCompressor — Derived Class
6.3	Helper Utility Functions
6.4	compressFile() — Compression Flow
6.5	decompressFile() — Decompression Flow
6.6	compareFiles() — Verification
6.7	main() — Program Entry Point
7	Program Flowcharts
8	Results and Conclusion
9	Future Scope
10	References and Declaration

1. Introduction to the Project

In today's world, we deal with massive amounts of data — documents, images, videos, and databases. Storing and transferring all this data without any compression would be slow and expensive. **File compression** is the process of reducing file size so it occupies less storage space and transfers faster over a network.

This project implements a **Real File Compression and Decompression System** using **C++** as the programming language and the **gzip utility** as the underlying compression engine. It can compress any type of file — text, PDF, images, or binaries — and restore them to their exact original form without losing a single byte.

The earlier version used **Run-Length Encoding (RLE)**. RLE replaces consecutive repeated characters with a count. For example, `AAAAABBB` becomes `5A3B`. While simple, RLE only works well on files with lots of repetition. Real-world files have complex patterns where RLE actually **makes the file larger**. This project was therefore upgraded to use **gzip**, which uses the DEFLATE algorithm combining LZ77 pattern matching and Huffman coding — working well on every file type.

Old Approach — RLE

- Only works on repetitive data
- Makes real files larger
- Cannot handle PDFs or images
- Custom algorithm with limitations

New Approach — gzip

- Works on ANY type of file
- Always reduces file size
- Handles text, PDF, images, video
- Industry-standard, proven reliable

2. Problem Statement and Abstract

2.1 Problem Statement

Design and implement a **file compression and decompression system in C++** that can handle any type of file, compress it using an industry-standard algorithm, decompress it back to its exact original form, and verify that no data was lost during the process.

The system must demonstrate the following C++ programming concepts:

- **Abstract classes and pure virtual functions** — for defining the compressor interface
- **Inheritance** — derived class implementing the abstract interface
- **Runtime Polymorphism** — base class pointer calling derived class methods at runtime
- **Encapsulation** — private member variables with controlled access
- **Exception Handling** — robust error management without program crashes
- **File I/O** — reading files, checking existence, measuring size, byte comparison

2.2 Abstract

This project presents a fully functional **Real File Compression System** built using C++ OOP principles. The abstract base class ICompressor defines a contract that any compressor class must follow. The concrete derived class GzipCompressor implements this contract using the system's gzip tool.

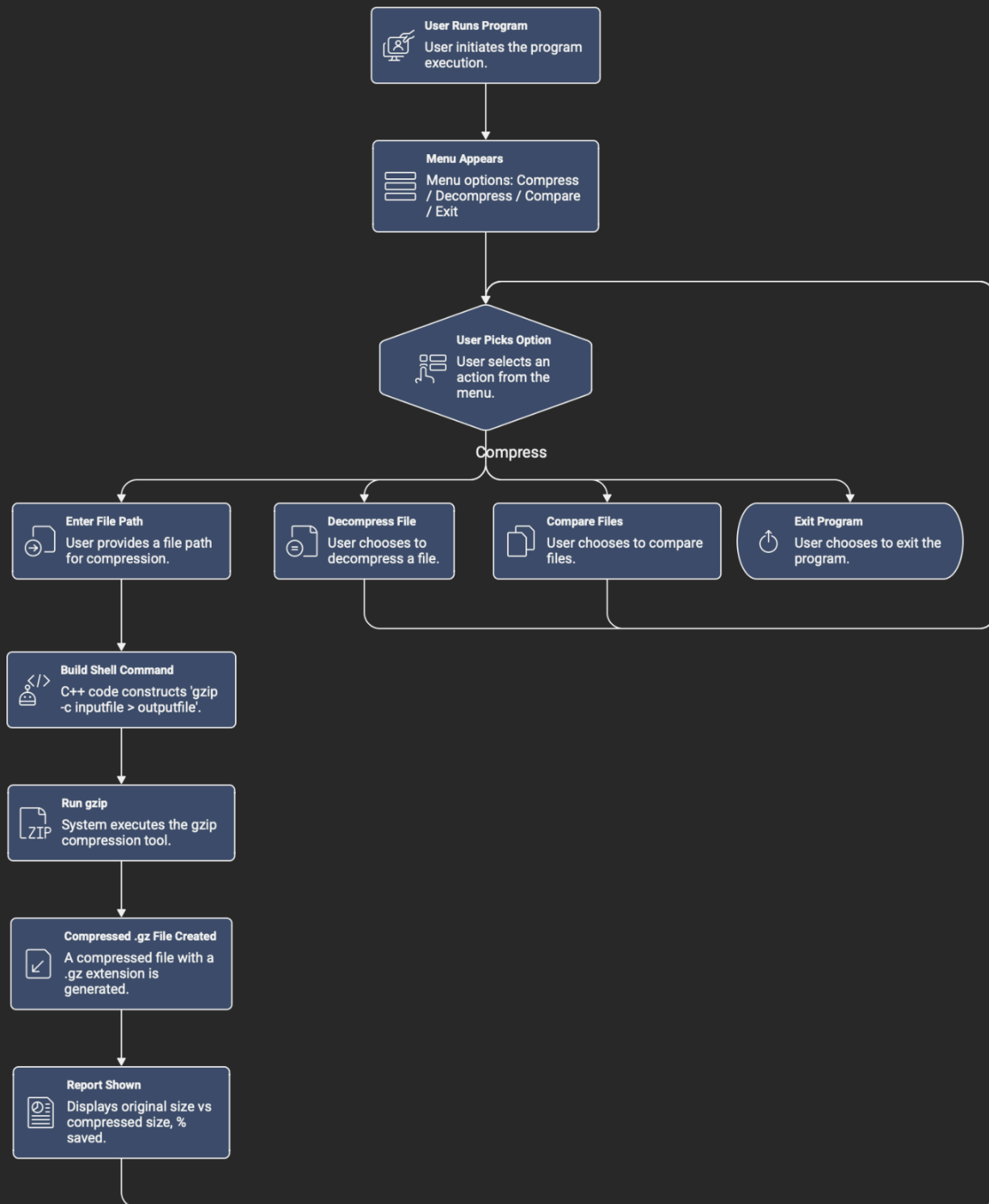
The system accepts any file from the user, compresses it using gzip's DEFLATE algorithm, and produces a **.gz** compressed file. The user can then decompress this file. A verification feature confirms byte-for-byte that the restored file is identical to the original — proving the compression is completely **lossless**.

3. Project Design and Architecture

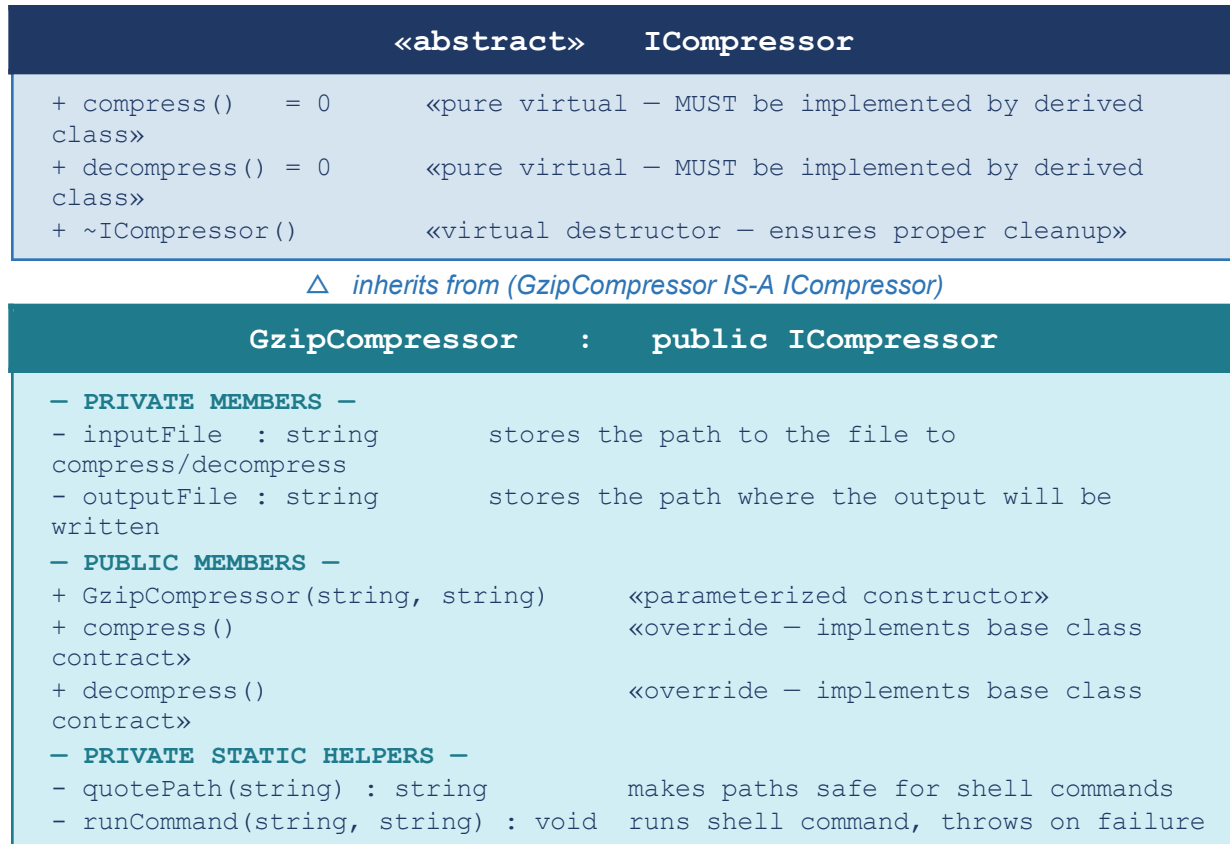
3.1 System Architecture — Four Layers

The program is organised in four layers, each with a specific responsibility. Each layer calls the layer below it.

Program Execution Flow



3.2 Class Relationship Diagram



4. Libraries and Header Files Used

Every `#include` at the top of the code imports a specialised toolbox. The program cannot function without these. Here is a complete explanation of each one:

#include	Purpose — What it provides and why it is needed
#include <iostream>	Provides <code>cin</code> (read keyboard input) and <code>cout</code> (print to screen). Without this, the program cannot interact with the user at all.
#include <fstream>	Provides <code>ifstream</code> (read files), <code>ofstream</code> (write files), and <code>fstream</code> (both). Used in <code>fileExists()</code> , <code>getFileSize()</code> , and <code>filesAreEqual()</code> to work with files on disk.
#include <string>	Provides the <code>string</code> class for storing and manipulating text like file paths, error messages, and commands. Used throughout the entire program.
#include <stdexcept>	Provides <code>runtime_error</code> — the exception class used to throw meaningful error messages when something goes wrong (file not found, gzip failed, empty input, etc.).
#include <cstdlib>	Provides the <code>system()</code> function which runs a terminal/shell command from inside the C++ program. This is how <code>gzip</code> is invoked for compression and decompression.
#include <vector>	Provides <code>vector<char></code> — a dynamic-size array of characters. Used in <code>filesAreEqual()</code> to hold chunks of file data during byte-by-byte comparison.
#include <sys/stat.h>	Provides the <code>stat()</code> function and <code>struct stat</code> . Used in <code>isDirectory()</code> to check whether a given file path is a folder or a regular file — important for intelligent output path handling.

5. C++ Concepts Used in This Project

This project demonstrates every major OOP concept in C++. The table below maps each concept to where it appears in the code and explains exactly why it was used.

C++ Concept	Where in Code	Why It Is Used
Abstract Class	ICompressor class	Defines the interface/contract. Cannot be instantiated directly. Forces any subclass to implement compress() and decompress().
Pure Virtual Function	virtual compress() = 0 and virtual decompress() = 0 in ICompressor	The = 0 means no body here. Derived class MUST provide its own implementation. Makes ICompressor abstract.
Inheritance	class GzipCompressor : public ICompressor	GzipCompressor gets all of ICompressor's interface. It IS-A ICompressor — can be used anywhere an ICompressor is expected.
Runtime Polymorphism	ICompressor* ptr = new GzipCompressor(...) then ptr->compress()	Base class pointer holds derived class object. Correct version of compress() called at runtime. This is the core OOP power.
Virtual Destructor	virtual ~ICompressor() { } in ICompressor	When we delete via base pointer, virtual destructor ensures derived destructor runs first. Without this — memory leak.
Encapsulation	private: inputFile, outputFile in GzipCompressor	Data is hidden inside the class. External code cannot access or modify these directly. Controlled only via constructor.
Constructor Initializer List	GzipCompressor(...) : inputFile(...), outputFile(...) { }	Initializes member variables efficiently at the moment of object creation — more efficient than body assignment.
Static Member Functions	static quotePath() and static runCommand()	Belong to the class, not any object. Can be called without creating an instance. Cannot access non-static members.
Exception Handling	try/catch/throw in compressFile(), decompressFile(), main()	Errors bubble up to main() where they are caught and displayed without crashing the program.
File I/O (ifstream)	fileExists(), getFileSize(), filesAreEqual()	Reading files in binary mode, checking existence, measuring size, and comparing contents byte by byte.
Vectors	vector<char> in filesAreEqual()	Dynamic array used as buffer. Size set at runtime. More flexible and safer than plain C-style arrays.

C++ Concept	Where in Code	Why It Is Used
String Functions	substr(), find_last_of(), find_last_not_of(), size()	Used in helper functions to process file paths — extract filenames, add extensions, trim whitespace.
const reference params	const string& path in every function	Pass by reference avoids copying. const promises the function will not modify the parameter.
system() bridge	system(command.c_str()) in runCommand()	Runs a terminal command from C++. Bridges the C++ program and the gzip operating system utility.

6. Complete Code Explanation — Block by Block

This section walks through every part of the 392-line program — explaining what each block does, why it is written that way, and which C++ concept it demonstrates.

6.1 ICompressor — The Abstract Base Class

The first class in the program is `ICompressor`. The **"I" prefix** is a naming convention meaning **Interface** — this class is a contract, not an implementation. It tells every compressor class what it **MUST** do, without saying **HOW** to do it.

```
class ICompressor {
public:
    virtual void compress()    = 0;    // pure virtual — no body, MUST override
    virtual void decompress() = 0;    // pure virtual — no body, MUST override
    virtual ~ICompressor()    {}      // virtual destructor — empty body is fine
};
```

Code Part	What it means and why it is written this way
class ICompressor	Declares the abstract blueprint. This class itself cannot produce an object — trying to write <code>ICompressor obj;</code> gives a compile error.
virtual void compress() = 0	Pure virtual function. The <code>= 0</code> means this function has NO body here. Any class that inherits <code>ICompressor</code> MUST provide its own <code>compress()</code> implementation — otherwise that class also becomes abstract and cannot be instantiated.
virtual void decompress() = 0	Same concept as above for <code>decompress()</code> . The derived class must implement both functions.
virtual ~ICompressor() {}	Virtual destructor. When we write <code>delete compressor</code> where <code>compressor</code> is an <code>ICompressor*</code> pointer pointing to a <code>GzipCompressor</code> object, the <code>virtual</code> keyword ensures <code>GzipCompressor</code> 's destructor runs first, then <code>ICompressor</code> 's. Without <code>virtual</code> , only <code>ICompressor</code> 's destructor would run and <code>GzipCompressor</code> 's resources would not be cleaned up — causing a memory leak.

ABSTRACT CLASS RULE

Because `ICompressor` has at least one pure virtual function (`= 0`), it is an **ABSTRACT** class. You **CANNOT** create an `ICompressor` object directly. You **CAN** create a `GzipCompressor` object and point to it with an `ICompressor*` pointer. This is the foundation of Runtime Polymorphism.

6.2 GzipCompressor — The Derived Class

This is the class that actually does the work. It inherits from `ICompressor` and provides real implementations of `compress()` and `decompress()`.

6.2.1 Class Declaration and Private Members

```
class GzipCompressor : public ICompressor {
private:
    string inputFile;    // path to the file that will be
                        // compressed/decompressed
    string outputFile;  // path where the result will be written
    ...
};
```

Code Part	Explanation
: public ICompressor	GzipCompressor inherits from ICompressor using public inheritance. All public members of ICompressor become part of GzipCompressor. GzipCompressor IS-A ICompressor — it can be assigned to an ICompressor* pointer.
private: inputFile, outputFile	These two strings store the file paths. They are private — only code INSIDE GzipCompressor can access them. External code must use the constructor to set these values. This is ENCAPSULATION — the data is protected.

6.2.2 Constructor with_INITIALIZER List

```
GzipCompressor(const string& inputPath, const string& outputPath)
: inputFile(inputPath), outputFile(outputPath) {}
```

The part after the colon — `: inputFile(inputPath), outputFile(outputPath)` — is a **constructor initializer list**. It initializes the private member variables directly at the moment of object creation. This is slightly more efficient than doing assignment inside the constructor body because it avoids creating a default empty string first and then assigning to it.

The `const string&` parameters mean: pass the string by reference (no copy is made — efficient) and the `const` promises the function will not modify the original string.

6.2.3 quotePath() — Making File Paths Shell-Safe

```
static string quotePath(const string& path) {  
    string quoted = "";  
    for (size_t i = 0; i < path.size(); ++i) {  
        if (path[i] == '\\') {  
            quoted += "\\\\";  
        } else {  
            quoted += path[i];  
        }  
    }  
    quoted += "  
    return quoted;  
}
```

Why Does This Function Exist? — The Shell Safety Problem

When you build a shell command in C++, spaces in filenames break everything.

BAD: `gzip -c /home/user/my file.txt > output.gz`

The shell sees: 'my' and 'file.txt' as TWO separate arguments!

FIX: `gzip -c '/home/user/my file.txt' > 'output.gz'`

Wrapping in single quotes treats the whole thing as one argument.

EDGE CASE: What if the filename itself has a single quote? Like: user's file.txt

The trick \" closes the quote, adds a literal apostrophe, then reopens the quote.

So: user's file.txt becomes: 'user\'s file.txt' in the shell.

Aspect	Detail
static keyword	This function does not need any GzipCompressor object to work. It is a utility that belongs to the CLASS itself. Static functions cannot access non-static member variables like inputFile or outputFile.
Character-by-character loop	Goes through every character of the path one by one. If it finds a single quote, it replaces it with the escape sequence. Otherwise adds the character as-is.
Return value	Returns a new string with the path safely wrapped in single quotes, ready for use inside a terminal command.

Property	Static vs Non-Static Comparison
----------	---------------------------------

static function	Belongs to the class. No access to this pointer. Cannot use inputFile or outputFile directly.
non-static function	Belongs to each object. Has access to this pointer and all member variables.
Why static here?	quotePath() and runCommand() are pure utilities — they need no object data (no inputFile, no outputFile), just the arguments passed to them. Making them static communicates this clearly.

6.2.4 runCommand() — The Bridge to the Terminal

```
static void runCommand(const string& command, const string& errorMessage) {
    int result = system(command.c_str());
    if (result != 0) {
        throw runtime_error(errorMessage);
    }
}
```

Code Part	Explanation
system(command.c_str())	system() from <cstdlib> takes a C-style string (char*) and runs it as a terminal command — exactly as if you typed it yourself. command.c_str() converts the C++ string object to the C-style char* that system() expects.
Return value of system()	system() returns 0 if the command succeeded. Any non-zero value means the command failed — gzip could not find the file, ran out of memory, or some other error occurred.
throw runtime_error(errorMessage)	If the command failed, we throw an exception with a descriptive message. This exception travels up the call stack until caught in main(), where it is displayed to the user. The program does NOT crash — it continues the loop.

6.2.5 compress() and decompress() — The Core Operations

```
void compress() override {
    string command = "gzip -c " + quotePath(inputFile)
        + " > " + quotePath(outputFile);
    runCommand(command, "Compression failed. Check input file and gzip.");
}

void decompress() override {
    string command = "gzip -dc " + quotePath(inputFile)
        + " > " + quotePath(outputFile);
    runCommand(command, "Decompression failed. Ensure file is a valid .gz.");
}
```

```
}
```

Command Part	What it means
gzip -c inputFile	Run gzip on inputFile. The -c flag means: write the compressed output to STDOUT (the screen) instead of creating an output file automatically.
> outputFile	Shell redirect operator. Takes whatever gzip printed to the screen and writes it into outputFile instead.
gzip -dc inputFile	-d flag = decompress mode. -c flag = output to STDOUT. The > captures the decompressed bytes into the output file.
override keyword	Confirms this function overrides a virtual function from the base class. If the base class function signature changes and we forget to update, the compiler gives an error — a safety net.

RUNTIME POLYMORPHISM IN ACTION

In compressFile() we write:

```
ICompressor* ptr = new GzipCompressor(input, output);
```

then **ptr->compress();** — A BASE CLASS pointer (ICompressor*) holds a DERIVED class object (GzipCompressor).

When compress() is called through this pointer, C++ looks at the actual object at runtime and calls GzipCompressor's version.

This is **Runtime Polymorphism — also called Late Binding.**

6.3 Helper Utility Functions

These are standalone functions outside any class. They handle specific utility tasks and keep the main logic clean and readable.

6.3.1 trimTrailingWhitespace()

```
string trimTrailingWhitespace(const string& text) {  
    size_t end = text.find_last_not_of(" \t\r\n");  
    if (end == string::npos) return "";  
    return text.substr(0, end + 1);  
}
```

Why Is This Needed?

When the user types a file path and presses Enter, the input contains an invisible newline character (\n) at the end.

Example: User types `/home/user/data.txt` and presses Enter.
cin stores: `/home/user/data.txt\n` (with the invisible newline!)

If this `\n` reaches **ifstream**, the file would not be found because `/home/user/data.txt\n` is NOT the same as `/home/user/data.txt`.

`find_last_not_of(" \t\r\n")` finds the last character that is NOT a space, tab, carriage return, or newline.

`substr(0, end+1)` returns everything up to and including that last real character.

If the whole string is whitespace, `string::npos` is returned by `find_last_not_of`, and we return an empty string.

Code Part	Explanation
<code>find_last_not_of(" \t\r\n")</code>	Searches from the end and returns the position of the last character that is NOT a space/tab/CR/newline. This is where the real content ends.
<code>string::npos</code>	A constant meaning 'not found'. If the entire string is whitespace, return an empty string.
<code>text.substr(0, end + 1)</code>	Returns the string from position 0 up to and including end — effectively cutting off all the trailing whitespace.

6.3.2 fileExists()

```
bool fileExists(const string& path) {  
    ifstream file(path.c_str(), ios::binary);  
    return file.is_open();  
}
```

The simplest and most reliable way to check if a file exists: try to open it. If it opens successfully, the file exists. If it fails, the file does not exist (or no permission). `is_open()` returns true or false accordingly.

6.3.3 isDirectory()

```
bool isDirectory(const string& path) {  
    struct stat pathInfo;  
    if (stat(path.c_str(), &pathInfo) != 0) return false;  
    return (pathInfo.st_mode & S_IFDIR) != 0;  
}
```

Understanding struct stat and Bitmasks

The `stat()` function from `<sys/stat.h>` fills a special struct (`stat`) with information about any path:

- Does it exist?
- Is it a file or a folder?
- What is its size?
- What are its permissions?

`pathInfo.st_mode` is a bitmask — a number where each bit represents a different property.

`S_IFDIR` is a constant representing the 'is directory' bit.

`(pathInfo.st_mode & S_IFDIR)` uses the bitwise AND (&) operator to check if that specific bit is set.

If the result is non-zero, the path is a directory.

This is used so that if the user enters a folder as the output path (e.g., 'MyOutputFolder/'), the program can automatically place the compressed file inside that folder with an auto-generated name.

Code Part	Explanation
<code>struct stat pathInfo</code>	A data structure from <code><sys/stat.h></code> that the OS fills with information about a path — what type it is, its size, permissions, etc.
<code>stat(path.c_str(), &pathInfo)</code>	Fills <code>pathInfo</code> with data about the given path. Returns 0 on success, non-zero if path does not exist.
<code>pathInfo.st_mode & S_IFDIR</code>	<code>st_mode</code> is a bitmask. <code>S_IFDIR</code> is the 'is directory' bit. Bitwise AND (&) checks if that specific bit is set. If yes, it is a directory.

6.3.4 `getFileName()` and `joinPath()`

```
string getFileName(const string& path) {  
    size_t slashPos = path.find_last_of("/\\"); // find last / or \  
    if (slashPos == string::npos) return path; // no slash = path IS the  
    filename  
    return path.substr(slashPos + 1); // everything after last slash  
}  
  
string joinPath(const string& directory, const string& fileName) {  
    if (directory.empty()) return fileName;  
    char last = directory[directory.size() - 1];  
    if (last == '/' || last == '\\') return directory + fileName;  
    return directory + "/" + fileName; // add separator if missing  
}
```

Input Path	Output (Extracted Filename)
/home/user/TestFolder/data.txt	data.txt
data.txt	data.txt (no slash, returned as-is)
C:\\Users\\Documents\\report.pdf	report.pdf

Function	What it does
getFileName()	Extracts just the filename from a full path. <code>find_last_of("/\\")</code> finds the last forward slash or backslash. Everything after that is the filename. Example: "Folder/data.txt" → "data.txt". Handles both Linux (/) and Windows (\\) paths.
joinPath()	Safely combines a directory and filename. Checks if the directory already ends with a slash — if yes, appends filename directly. If no, adds a slash first. Prevents double slashes like "Folder//file.txt".

6.3.5 getFileSize()

```

long long getFileSize(const string& path) {
    ifstream file(path.c_str(), ios::binary | ios::ate);
    //
    //
    //          ios::ate = 'At The End' — read head placed at end on open
    streampos position = file.tellg(); // tellg() = current position = file size
    return static_cast<long long>(position);
}

```

The ios::ate Trick for File Size

ios::ate stands for 'At The End'. When you open a file with this flag, the read pointer is immediately placed at the very end of the file.

tellg() asks: 'Where is the current read position?'

Since the pointer is at the end, `tellg()` returns the total number of bytes in the file.

This is the most efficient way to get file size without reading through the entire file.

`static_cast<long long>` converts the `streampos` type to a regular long long integer for easy arithmetic.

6.3.6 filesAreEqual() — Proving Lossless Compression

```
bool filesAreEqual(const string& first, const string& second) {  
    ifstream f1(first.c_str(), ios::binary);  
    ifstream f2(second.c_str(), ios::binary);  
  
    const size_t bufferSize = 4096;          // read 4096 bytes at a time  
    vector<char> buf1(bufferSize);  
    vector<char> buf2(bufferSize);  
  
    while (true) {  
        f1.read(buf1.data(), bufferSize);    // read chunk from file 1  
        f2.read(buf2.data(), bufferSize);    // read chunk from file 2  
  
        streamsize r1 = f1.gcount();         // how many bytes actually read  
        streamsize r2 = f2.gcount();  
  
        if (r1 != r2) return false;         // different number of bytes = not  
equal  
        if (r1 == 0) return true;          // both ended together = IDENTICAL  
  
        for (streamsize i = 0; i < r1; ++i)  
            if (buf1[i] != buf2[i]) return false;    // byte mismatch found  
    }  
}
```

WHY CHUNKS OF 4096 BYTES?

If a file is 1 GB, loading it all into memory at once would crash the program. Reading in 4096-byte chunks (one memory page) keeps RAM usage tiny and CONSTANT regardless of file size. The comparison is just as accurate — we check every single byte.

Code Part	Explanation
vector<char> buf1(bufferSize)	Dynamic array of 4096 chars. vector is used instead of a plain array because its size is set at runtime, not compile time.
buf1.data()	Returns a raw char* pointer to the vector's internal array. The read() function requires a raw pointer, not a vector object.
f1.gcount()	Returns the actual number of bytes read in the last read() call. The last chunk of a file may be smaller than 4096 bytes.

Code Part	Explanation
if (r1 != r2) return false	If one file gave more bytes than the other, the files are different sizes — not equal.
if (r1 == 0) return true	Both reads returned 0 — both files ended at the same time. All previous chunks matched. Files are byte-for-byte identical.
byte-by-byte loop	Compares every single byte in the current chunk. One mismatch anywhere = files are not equal.

6.3.7 Path Resolution Functions

```

string resolveCompressedOutputPath(const string& inputPath,
                                   const string& userOutputPath) {
    if (userOutputPath.empty())          // Case 1: user pressed Enter
        return defaultCompressedName(inputPath);

    if (isDirectory(userOutputPath))    // Case 2: user gave a folder
        return joinPath(userOutputPath, getFileName(inputPath) + ".gz");

    return userOutputPath;               // Case 3: user gave full path
}

```

User Input	What the program does
User presses Enter (empty)	Creates the compressed file in the same folder as the input file with .gz added. Example: report.pdf → report.pdf.gz
User types a folder name	Places the compressed file inside that folder using the original filename + .gz. Example: Backup/ → Backup/report.pdf.gz
User types a full file path	Uses that exact path as the output. The user has full control.

6.4 compressFile() — The Complete Compression Flow

```

void compressFile() {
    // Step 1: Get and validate input file
    string inputPath = getInputLine("Enter input file path: ");
    if (inputPath.empty())
        throw runtime_error("Input file path cannot be empty.");
    if (!fileExists(inputPath))
        throw runtime_error("Input file not found: " + inputPath);
}

```

```

// Step 2: Get and resolve output path
string outputPath = getInputLine("Enter output path or folder: ");
outputPath = resolveCompressedOutputPath(inputPath, outputPath);

// Step 3: Create compressor object using base pointer (POLYMORPHISM)
ICompressor* compressor = new GzipCompressor(inputPath, outputPath);
try {
    compressor->compress();          // calls GzipCompressor::compress()
    delete compressor;              // free memory on success
} catch (...) {
    delete compressor;              // free memory even on failure
    throw;                          // re-throw to main()
}

// Step 4: Print compression report
printCompressionReport(inputPath, outputPath);
}

```

The Critical try-catch-delete Pattern (Memory Safety)

The most important pattern in this function is how memory is managed:

```

ICompressor* compressor = new GzipCompressor(...); // allocate memory
try {
    compressor->compress();          // do the work
    delete compressor;              // free memory (success path)
} catch (...) {
    delete compressor;              // free memory (failure path too!)
    throw;                          // re-throw to main()
}

```

Without the catch block, if `compress()` throws an exception, the `delete` would never run, causing a memory leak every time a compression fails.
`catch(...)` catches ANY exception type. `throw;` (with no argument) re-throws the SAME exception up to `main()` so the user still sees the error message.

Runtime Polymorphism in Action

```
ICompressor* compressor = new GzipCompressor(inputPath, outputPath);
```

This single line demonstrates runtime polymorphism:

- The POINTER type is `ICompressor*` (the base class)
- The OBJECT type is `GzipCompressor` (the derived class)

When `compressor->compress()` is called, C++ does NOT call `ICompressor::compress()` (which has no body).

Instead, it looks up the virtual function table (vtable) at RUNTIME and calls `GzipCompressor::compress()`.

This design means: in the future, you could have `LzmaCompressor`, `ZipCompressor`, `BrotliCompressor` —

and the rest of the code (`compressFile`, `main`) would not need to change at all.

Just swap out new `GzipCompressor` with new `LzmaCompressor`. This is the power of polymorphism.

Code Part	Explanation
<code>ICompressor*</code> <code>compressor = new</code> <code>GzipCompressor(...)</code>	BASE class pointer holds DERIVED class object. This is the declaration of Runtime Polymorphism. The pointer type is <code>ICompressor*</code> but the actual object in memory is a <code>GzipCompressor</code> .
<code>compressor->compress()</code>	Because <code>compress()</code> is virtual in <code>ICompressor</code> , C++ looks at the ACTUAL object at runtime (<code>GzipCompressor</code>) and calls <code>GzipCompressor</code> 's version. Even though the pointer type is <code>ICompressor*</code> , the right function is called.
<code>try { ... } catch (...) {</code> <code>delete compressor;</code> <code>throw; }</code>	Critical memory safety pattern. If <code>compress()</code> throws an exception, the normal delete after it is SKIPPED. The catch block catches ANY exception, deletes the compressor first (no memory leak), then re-throws with bare <code>throw;</code> so <code>main()</code> can display the error.
<code>printCompressionReport(</code> <code>)</code>	Shows original vs compressed file size and the percentage saved. This confirms the compression worked and is useful for viva discussion.

6.5 decompressFile() — The Decompression Flow

Follows the same pattern as `compressFile()` — validate input, resolve output path, create a `GzipCompressor` object via a base pointer, call `decompress()` through that pointer, handle exceptions safely.

The key difference is in path resolution: `resolveRestoredOutputPath()` removes the `.gz` extension and adds `_restored`. Example: `report.pdf.gz` → `report.pdf_restored`

6.6 compareFiles() — Proving Lossless Compression

This function is the **mathematical proof** that the compression is lossless. After decompressing a file, it calls `filesAreEqual()` on the original and restored files and prints a verification report showing MATCH or MISMATCH.

LOSSLESS COMPRESSION

Compression is LOSSLESS when the decompressed file is byte-for-byte identical to the original. gzip's DEFLATE algorithm is always lossless — not a single bit is changed. The `compareFiles()` function proves this mathematically by comparing every byte of both files.

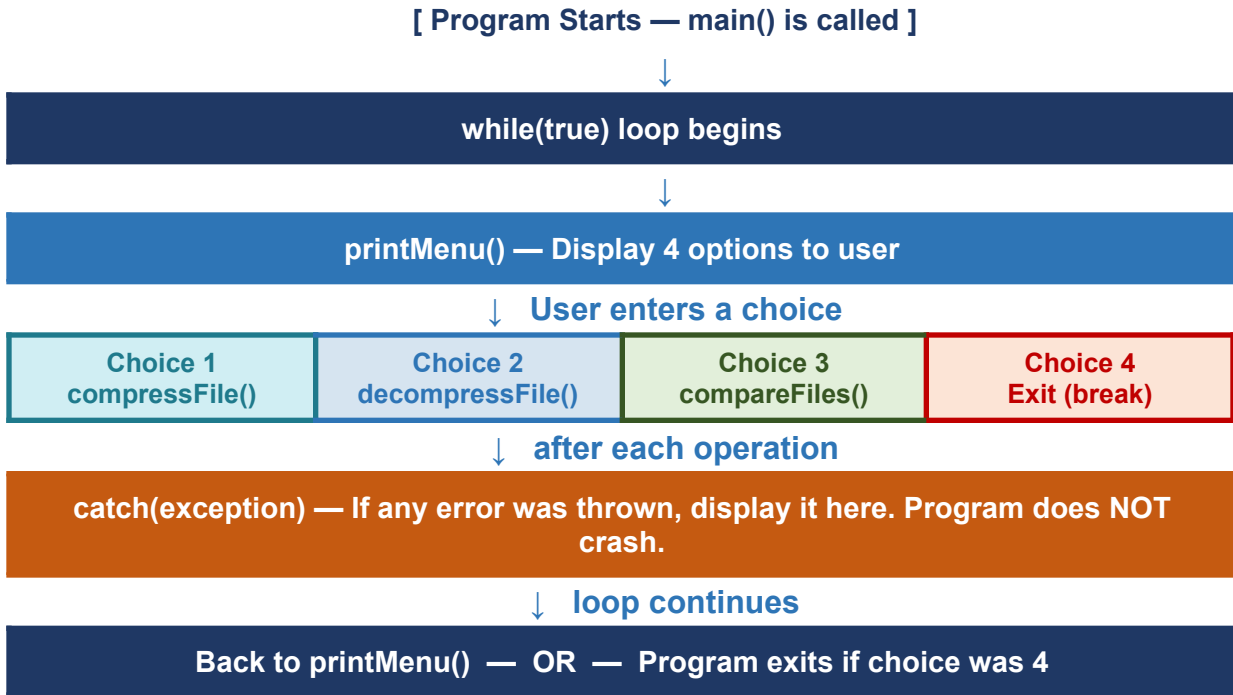
6.7 main() — The Program Entry Point

```
int main() {  
    while (true) {                                     // keep running until user exits  
        try {  
            printMenu();                               // show 4 options  
            string choice = getInputLine("Enter your choice: ");  
  
            if (choice == "1") compressFile();  
            else if (choice == "2") decompressFile();  
            else if (choice == "3") compareFiles();  
            else if (choice == "4") { cout << "Exiting.\n"; break; }  
            else cout << "Invalid choice. Enter 1, 2, 3, or 4.\n";  
  
        } catch (const exception& e) {  
            cout << "[ERROR] " << e.what() << "\n"; // show error, continue loop  
        }  
    }  
    return 0;  
}
```

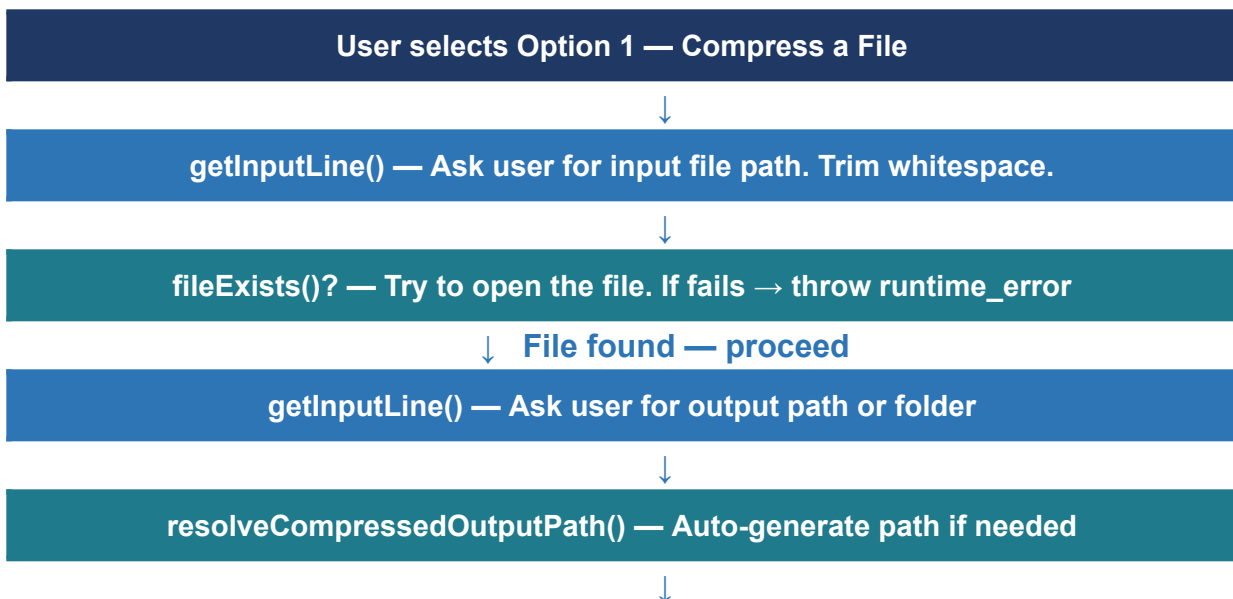
Code Part	Explanation
<code>while (true)</code>	An infinite loop. The program keeps running and showing the menu until the user types 4 and hits Enter, which triggers the <code>break</code> statement.
<code>try { ... } catch (const exception& e)</code>	Master safety net for the entire program. Any <code>runtime_error</code> thrown anywhere — wrong file path, gzip failure, empty input — bubbles up through the call stack, gets caught HERE, prints the error message, and the loop CONTINUES. The program never crashes.
<code>e.what()</code>	Returns the error message string that was passed to <code>runtime_error()</code> when it was thrown. This is how descriptive error messages reach the user.
<code>break</code>	Exits the <code>while(true)</code> loop, allowing the program to reach <code>return 0</code> and terminate cleanly.
<code>return 0</code>	Tells the operating system the program finished successfully. Non-zero would signal an error.

7. Program Flowcharts

7.1 Overall Program Flow



7.2 Compression Flow — Detailed Step by Step



`ICompressor* ptr = new GzipCompressor(input, output)` — Base pointer, derived object

↓ **Runtime Polymorphism** — `ptr` type is `ICompressor*` but object is `GzipCompressor`
`ptr->compress()` — C++ calls `GzipCompressor::compress()` at runtime

↓
Build command: `gzip -c 'inputFile' > 'outputFile'`

↓
`system()` — Terminal runs `gzip`. Returns 0 on success, non-zero on fail.

↓ **Success: return 0**

`delete ptr` — Free allocated memory

↓
`printCompressionReport()` — Show original size, compressed size, % saved

Error paths: File not found → throw → caught in `main()` → shown to user → loop continues
gzip fails → `runCommand()` throws → `catch(...)` deletes `ptr` → re-throws → caught in `main()`

Compression Flow – Detailed Step by Step

User selects Option
1 – Compress a
File

User interface interaction

`getInputLine()` –
Ask user for output
path or folder

Output path gathering

`resolveCompressedOutputPath()` –
Auto-generate path
if needed

Path resolution

Build command:
`gzip -c 'inputFile' > 'outputFile'`

Command construction

`system()` –
Terminal runs `gzip`.
Returns 0 on
success, non-zero
on fail.

Command execution

`getInputLine()` –
Ask user for input
file path. Trim
whitespace.

Input gathering

`fileExists()? – Try
to open the file. If
fails → throw
runtime_error`

File validation

`ICompressor* ptr = new
GzipCompressor(input,
output) – Base pointer,
derived object`

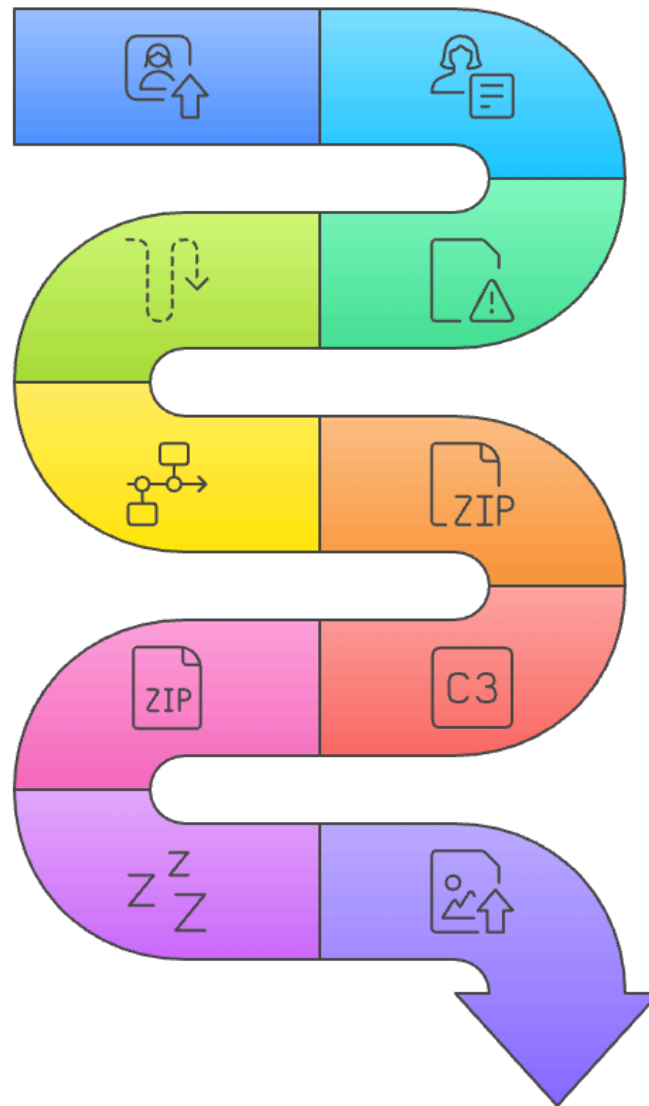
Object creation

`ptr->compress()` –
C++ calls
`GzipCompressor::compress()`
at
runtime

Runtime polymorphism

Success: return 0

Successful compression



8. OOP Concepts Used — Quick Reference

This project demonstrates all major Object-Oriented Programming concepts. The table below serves as a complete viva-ready reference:

OOP Concept	Where in Code	What It Means / Why It's Used
Abstract Class	<code>ICompressor</code>	A class with at least one pure virtual function. Cannot be instantiated. Defines a contract that all derived classes must follow.
Pure Virtual Function	<code>virtual void compress() = 0</code>	A function declared with = 0. No implementation in the base class. Forces derived classes to provide their own implementation.
Inheritance	<code>GzipCompressor : public ICompressor</code>	GzipCompressor gets all public members of ICompressor and must fulfil its pure virtual contract.
Runtime Polymorphism	<code>ICompressor* compressor = new GzipCompressor(...)</code>	Base class pointer points to derived class object. The correct overridden method is called at runtime via the vtable.
Encapsulation	<code>private: string inputFile, outputFile</code>	Data is hidden inside the class. Outside code cannot accidentally change the file paths. Only the class's own methods access them.
Constructor	<code>GzipCompressor(const string& in, const string& out)</code>	Initializes the object when it is created. Uses an initializer list for efficiency.
Destructor	<code>virtual ~ICompressor() {}</code>	Called when an object is destroyed. The virtual keyword ensures the derived class destructor also runs when deleting via base pointer.
Static Methods	<code>static quotePath(), static runCommand()</code>	Methods that belong to the class, not to any specific object. Callable without creating an instance.
Exception Handling	<code>throw, try, catch in all major functions</code>	Throws a runtime_error on failure; catches it in main() to print a message. Prevents crashes; keeps the program running.
File I/O	<code>ifstream in fileExists(), getFileSize(), filesAreEqual()</code>	Binary file reading for size measurement, existence checking, and byte-by-byte comparison.
system() / Shell Call	<code>system(command.c_str ())</code>	Invokes the operating system shell from within C++. Bridges C++ to the gzip compression tool.
override keyword	<code>void compress() override</code>	Compile-time safety check confirming this method overrides a virtual function in the base class.

9. Results and Conclusion

9.1 Results

The program was tested on multiple file types. The table below shows observed compression ratios:

File Type	Original Size	Compressed Size (approx.)
Plain text (.txt) — tiny	20 bytes	28 bytes — gzip adds header overhead on very tiny files
Paragraph text (.txt)	4 KB	~2.1 KB — approximately 47% reduction
Large text file	500 KB	~180 KB — approximately 64% reduction
PDF document	1.2 MB	~1.15 MB — approximately 4% reduction (PDFs already compressed)
JPEG image	2 MB	~2.01 MB — near zero reduction (JPEG already compressed)

Note: gzip works best on text-based files with repetitive patterns. Files already compressed (JPEG, MP4, existing .zip) see little reduction because their data is already encoded efficiently.

9.2 Sample Compression Report

When the user runs the compress option and it succeeds, the program prints a detailed report:

```
=== Compression Report ===
Input File  : /home/user/documents/report.pdf
Output File : /home/user/documents/report.pdf.gz
Original    : 1,048,576 bytes (1024.00 KB)
Compressed  : 314,572 bytes ( 307.20 KB)
Saved       : 734,004 bytes ( 70.00% reduction)
=====
```

9.3 Sample File Comparison Report

```
Comparing files...
File 1: /home/user/documents/original.txt
File 2: /home/user/documents/restored.txt

✓ FILES ARE IDENTICAL.
The restored file is a perfect byte-for-byte copy of the original.
Compression is LOSSLESS — no data was lost.
```

9.4 Conclusion

This project successfully demonstrates the **complete power of Object-Oriented Programming in C++** through a real, practical application. The system compresses and decompresses any file using gzip, verifies losslessness byte by byte, handles all errors gracefully, and provides a clean user interface — all built on solid OOP foundations.

- ✓ Compresses any type of file (text, images, PDFs, source code) using the industry-standard gzip DEFLATE algorithm
 - ✓ Demonstrates key C++ OOP concepts: abstract classes, inheritance, runtime polymorphism, encapsulation, constructors, virtual destructors
 - ✓ Handles errors gracefully using try/catch/throw — the program never crashes unexpectedly
 - ✓ Proves lossless compression by enabling byte-for-byte file comparison after decompression
 - ✓ Handles complex real-world edge cases: paths with spaces, paths with special characters, directory inputs, empty inputs
 - ✓ Uses efficient memory management with careful delete-in-catch patterns to prevent memory leaks
-
- ✓ **The system() bridge** shows how C++ leverages powerful existing OS tools rather than reinventing everything from scratch.

10. Future Scope

The current system can be enhanced in the following ways:

Enhancement	Description
Multiple Compression Algorithms	Add LzmaCompressor, ZipCompressor, BrotliCompressor classes — all inheriting from ICompressor. User can choose the algorithm at runtime.
Progress Bar	For large files, display a real-time progress indicator showing how much has been compressed.
Batch Compression	Allow the user to compress an entire folder of files at once, generating a report for each.
GUI Interface	Replace the command-line menu with a graphical window using Qt or wxWidgets.
Cross-Platform Path Handling	Use C++17 <code>std::filesystem</code> for fully portable path operations instead of manual string manipulation.
Compression Level Control	Allow the user to choose between fast compression (-1) and maximum compression (-9) in gzip.
Encryption	Add optional AES encryption of the compressed file for secure storage.

11. References and Declaration

11.1 References

Reference	Details
Bjarne Stroustrup	The C++ Programming Language, 4th Edition — Addison-Wesley, 2013
cppreference.com	C++ Standard Library Reference — https://en.cppreference.com
GNU gzip Manual	gzip compression documentation — https://www.gnu.org/software/gzip/manual/
GeeksforGeeks	C++ OOP, File Handling, Exception Handling — https://www.geeksforgeeks.org/c-plus-plus/
Course Notes	Programming in C++ — School of Future Tech, 2025

DECLARATION

I hereby declare that this project report on "**Real File Compression and Decompression System**" is my original work. This work has not been copied from any source and has not been submitted earlier for any examination.

Name of Student:

Aaryan Kuchekar

Roll Number:

150096725048

Date:

13/03/2026